Mini Sentrix

Day 1 Proof Deck — Foundation & Architecture

A behaviour-aware Mini SOC built for the SRM network environment

---

TEAM 5

Date: 19 June 2026
Event: Mini Sentrix Hackathon, powered by Talenciaglobal
Team size: 4

Table of Contents

1. Executive Summary

Mini Sentrix is a four-layer Security Operations Center built across two physical machines and two virtual machines, designed from the ground up around a single architectural principle: a SOC that only logs events without remembering behaviour over time is not meaningfully different from no SOC at all.

This idea is the direct response to the structure of the hackathon's own brownie point riddles, all four of which point at the same underlying gap — detection systems that see individual events but never the pattern those events form across time. Rather than treat each day's brownie point as an isolated puzzle, the team has architected Day 1 around a shared, persistent correlation layer that every later layer will read from and write to. This is the project's core source of novelty: a stateful risk-scoring engine sitting between the SIEM and the response layer, intended to give Mini Sentrix memory that conventional rule-only detection lacks.

On Day 1, the team prioritised getting the perimeter detection layer fully operational and verified end-to-end before expanding to the remaining three layers, on the principle that a single layer working under real traffic is worth more than four layers stubbed out and untested.

2. Team & Role Ownership

Each member owns one SOC layer outright and additionally contributes to the shared correlation engine, so that every team member can speak to both their individual component and the system's unifying idea during Q&A.

Member	Primary Layer Owned	Correlation Engine Contribution
Member A	Perimeter & Detection (Kali attacker VM, attack scripting)	Supplies raw attack event data into the engine's event schema
Member B	Perimeter & Detection (target VM, WAF, Suricata config)	Defines what counts as a “notable” event worth correlating
Member C	SIEM & Visibility (Wazuh, ELK, dashboard)	Builds the visualisation layer for engine output / risk scores
Member D	Identity & Access + Automated Response (Keycloak, Shuffle)	Consumes engine risk scores to decide when a playbook fires

3. System Architecture

3.1 Physical / Virtual Topology

The system is split across two physical laptops to balance compute load and avoid resource contention with VM hypervisor overhead:

- Laptop 1 (M1 MacBook Air, 16GB, VMware Fusion): hosts the Kali Linux attacker VM and the Ubuntu target VM on a shared host-only network adapter. Suricata runs inside the Ubuntu VM in AF_PACKET mode, watching the host-only interface.
- Laptop 2 (Docker host): runs the central stack via Docker Compose — Wazuh indexer, manager and dashboard (SIEM), Keycloak (IAM), Shuffle (SOAR), and the custom correlation engine.
- The Wazuh agent on the Ubuntu VM ships logs to Laptop 2's Wazuh manager over the shared LAN, forming the only network dependency that crosses physical machines.

This split was chosen specifically to satisfy the SRM network constraints: all attack traffic and packet capture stay within a host-only virtual adapter on a single machine, so the university's restrictions on promiscuous mode and physical VLAN access never come into play. Nothing in the pipeline requires internet-facing ports or external tunnels.

3.2 The Four Mandatory Layers

Layer	Tool(s)	Role in Mini Sentrix
Perimeter & Detection	Suricata (IDS), ModSecurity + Nginx (WAF)	Detects malicious traffic against the target VM and raises the first signal
SIEM & Visibility	Wazuh + ELK Stack, Kibana	Aggregates logs from every layer and exposes them on a live dashboard
Identity & Access	Keycloak	Controls and audits access to internal services protected behind SSO/RBAC
Automated Response	Shuffle (SOAR)	Executes an automatic action once the correlation engine raises a qualifying alert

3.3 The Correlation & Risk-Scoring Engine

Sitting between the SIEM and the response layer is a lightweight custom service, intentionally kept small and auditable. Its job is the one the riddles keep pointing at: remembering what has already happened, rather than evaluating every event as if it is the first one the system has ever seen.

- Maintains short-term state per source IP / identity across a rolling time window, rather than treating every alert as independent.
- Assigns a rising risk weight to repeated or sequence-linked behaviour, instead of giving every alert equal severity by default.
- Exposes its output to both the dashboard (for visibility) and the SOAR layer (to decide whether automated response should trigger).

This engine is being introduced on Day 1 deliberately, even though it is not required for today's scope, so that the Day 2 through Day 4 brownie points can be implemented as natural extensions of one coherent subsystem rather than as three unrelated last-minute scripts.

4. SRM Network Constraint Handling

The architecture was deliberately constrained to avoid every category of tooling flagged as unreliable on the SRM network:

Constraint Avoided	How Mini Sentrix Avoids It
Raw packet sniffing on shared physical interfaces	All capture happens inside a host-only virtual adapter, not the physical NIC
External VPN tunnels	Zero traffic leaves the local environment; the only cross-machine link is LAN-local log shipping
Mid-hackathon large downloads	All Docker images and VM ISOs were pre-pulled on Day 1 morning before network congestion
Internet-facing port exposure	Every service binds to host-only or LAN-local interfaces only
Physical VLAN configuration	Not used; isolation is achieved entirely through VMware host-only networking

5. Tool Choice Justification

Every tool below was chosen against at least one named alternative. “It was easier to install” is not treated as sufficient justification within the team.

Layer	Chosen	Alternative Considered	Why Chosen
IDS	Suricata	Snort, Zeek	Runs cleanly in AF_PACKET mode on a virtual interface with no promiscuous-mode dependency, which Snort's default capture path is more sensitive to on restricted networks
WAF	ModSecurity + Nginx	Cloud WAF (Cloudflare, AWS WAF)	Fully local and Docker-deployable; cloud WAFs require internet-facing

Layer	Chosen	Alternative Considered	Why Chosen
			exposure, which is explicitly unreliable on SRM's network
SIEM	Wazuh + ELK	Splunk Free, Graylog	No external connectivity needed once deployed; Splunk's free tier has a daily ingestion cap unsuited to a multi-day live demo
IAM	Keycloak	Authelia	Native RBAC and SSO support gives richer audit/event data to feed the correlation engine than Authelia's lighter access-proxy model
SOAR	Shuffle	Custom Python scripts only	Gives a visual, demoable playbook builder for the live Proof Deck while still allowing custom logic where Shuffle's nodes fall short

6. Day 1 Progress & Verification

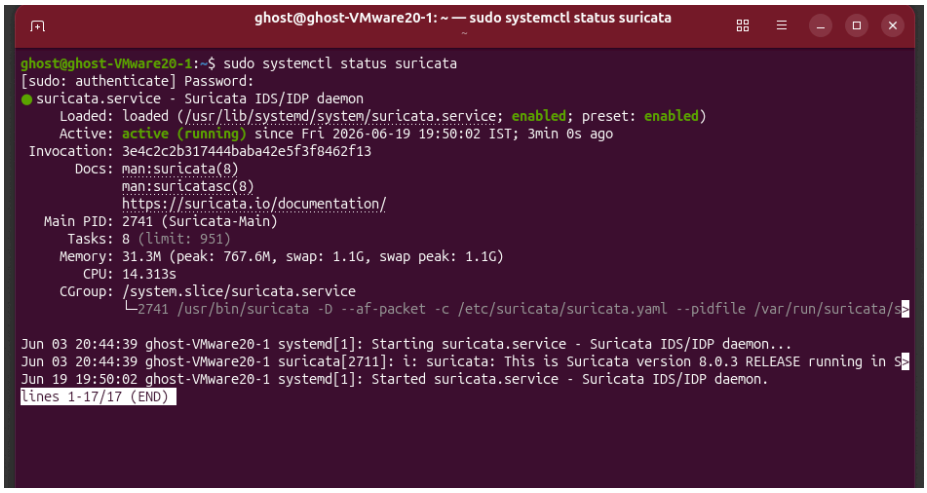
6.1 Completed Today

- Kali Linux (attacker) and Ubuntu (target) VMs provisioned on a shared VMware Fusion host-only network adapter
- Cross-VM connectivity verified — Kali and Ubuntu VMs can reach each other on the host-only segment
- Suricata installed and configured in AF_PACKET mode inside the Ubuntu target VM
- Suricata service confirmed active and running as a systemd-managed daemon (see verification evidence below)
- Docker images and installers for Wazuh, ELK, Keycloak, and Shuffle pre-pulled on the second laptop ahead of network congestion

6.2 In Progress / Carrying Into Day 2

- Wazuh manager deployment via Docker Compose on the second laptop
- Wazuh agent enrollment from the Ubuntu VM to the central manager
- ModSecurity + Nginx WAF deployment in front of the DVWA target application
- First end-to-end attack simulation (Nmap scan → Suricata alert → SIEM visibility)

6.3 Verification Evidence — Suricata Service Status



Suricata 8.0.3 running in AF_PACKET mode as an enabled systemd service inside the Ubuntu target VM, confirmed via systemctl status.

This output confirms the service is active (running), loaded from the standard systemd unit path, and bound to the AF_PACKET interface as required for SRM-compatible packet capture without promiscuous mode.

7. Day 1 Brownie Point — Decision

The team has opted not to attempt “The Sleepwalker's Diary” on Day 1. The riddle's decode — that a SIEM which logs without correlating across time is functionally blind to slow, multi-day attacker behaviour — is already the founding idea behind the correlation engine described in Section 3.3. Rather than rush a partial implementation today, the team is deliberately building the underlying state-tracking infrastructure first, so that the Day 1 idea can be implemented properly as part of a later brownie point once the core pipeline (Suricata → Wazuh → dashboard) is fully verified end-to-end.

The team remains on track to meet the minimum requirement of two implemented brownie points across the remaining three days.

8. Immediate Next Steps (Day 2 Entry Point)

- Complete Wazuh manager + agent enrollment and confirm the Ubuntu VM shows as an active agent
- Run the first Nmap scan from Kali against the Ubuntu target and trace the alert from Suricata into the Wazuh dashboard
- Stand up ModSecurity in front of DVWA

- Begin Keycloak deployment to start generating authentication event data for the Day 2 brownie point (“The Loyal Guard Who Lets Everyone In”)